

</>

The Anuj DECODED

</>

```

01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
21 };

```



```

01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver

```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

3871 · Count Commas in Range II

LeetCode · Daily Question

Medium

Math

Digit Counting

1. What Type of Question Is This?

This is a **digit-counting / range-formula problem**, in the same family as "count how many times digit X appears from 1 to n" problems. The number of commas a value needs depends *only on how many digits it has*, not on the digits' actual values. That's the key observation: instead of thinking about individual numbers, group them by **digit length** — all 1-digit numbers behave the same way, all 2-digit numbers behave the same way, and so on. With n up to 10^{15} , iterating number by number is completely off the table; the whole problem is really about turning an $O(n)$ loop into an $O(\log n)$ loop over *digit-length groups* instead.

2. What Is Given?

- An integer **n**.
- "Standard number formatting" rules for writing an integer as text:

Comma every 3 digits from the right	e.g. 1234567 is written 1,234,567
Fewer than 4 digits → no commas	e.g. 999 is written 999 (no comma)

3. What Do We Need to Find?

The **total** number of commas used across every integer written from 1 to n inclusive, when each one is written with standard comma formatting.

Function Signature

■ C++

```
class Solution {
public:
    long long countCommas(long long n) {

    }
};
```

Examples Walkthrough

Example 1 — used as the running example for every code sample below

```
Input:  n = 1002
Output: 3
```

Every number from 1 to 999 has 3 or fewer digits, so none of them use a comma. Only 1000, 1001, and 1002 are 4-digit numbers, and each 4-digit number gets exactly one comma ("1,000", "1,001", "1,002") — 3 numbers × 1 comma each = **3**.

Example 2 — no numbers reach 4 digits

```
Input:  n = 998
Output: 0
```

Every number from 1 to 998 has at most 3 digits, so by the rule ("fewer than 4 digits → no commas") none of them use any commas at all → **0**.

Constraints

- $1 \leq n \leq 10^{15}$

n can be as large as a quadrillion. Looping over every single integer from 1 to n is utterly infeasible (it would take far longer than a lifetime) — this problem **requires** a closed-form / grouped counting approach; it cannot be brute forced in any real sense.

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — Figure out how many commas a single number needs, purely from its digit count.** A number with d digits gets a comma after every group of 3 digits from the right, which works out to exactly $(d - 1) \div 3$ commas (integer division). Check it: $d=3 \rightarrow 0$, $d=4 \rightarrow 1$, $d=6 \rightarrow 1$, $d=7 \rightarrow 2$. Matches the stated rule exactly.
- **Step 2 — Realize the commas-per-number only depends on digit length, not the specific digits.** Every 4-digit number needs exactly 1 comma, whether it's 1000 or 9999. That means you never need to look at individual numbers — you only need to know *how many* numbers in $[1, n]$ have each possible digit length.
- **Step 3 — Count how many numbers of each digit length fall inside $[1, n]$.** All d -digit numbers live in the range $[10^{d-1}, 10^d - 1]$. If n is at least $10^d - 1$, that entire range of $9 \times 10^{d-1}$ numbers is included. If n falls in the middle of that range (n is itself a d -digit number), only $n - 10^{d-1} + 1$ of them are included, and you can stop after this group.
- **Step 4 — Sum contributions across digit-length groups.** $\text{total} = \sum \text{over } d \text{ of (how many } d\text{-digit numbers are in } [1, n]) \times ((d - 1) \div 3)$. Since n has at most 16 digits (10^{15} has 16), this sum has at most 16 terms — a tiny, fixed-size loop no matter how large n gets.

Which Algorithm Is Best?

Walking the digit-length groups from $d = 1$ upward, tracking the $[10^{d-1}, 10^d - 1]$ boundaries with simple multiplication, and stopping once the group's lower bound exceeds n , gives an $O(\log_{10} n)$ algorithm — at most ~16 iterations regardless of how large n is, all the way up to 10^{15} . This is the only approach of the three below that actually finishes in reasonable time on the full constraint range; both "brute force" tiers genuinely cannot complete for large n , no matter how much time you give them.

Approach 1 — Brute Force (Literally Format Every Number and Count Commas)

Idea: Translate the problem statement as directly as possible: for every integer from 1 to n, actually build its comma-formatted string exactly per the stated rule, then count how many commas ended up in it. This is the clearest possible statement of what the problem is asking, with zero mathematical insight required.

Steps

- For a given x, convert it to a plain digit string.
- Walk through the digits, inserting a comma just before any digit that has a multiple of 3 digits remaining to its right (this reproduces "comma after every 3 digits from the right").
- Count the comma characters in the resulting string.
- Repeat for every x from 1 to n, summing the comma counts.

Approach 1 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    // Literally builds the comma-formatted string for x, exactly per the problem's own rule.
    string withCommas(long long x) {
        string raw = to_string(x);
        string result;
        int len = raw.size();
        for (int i = 0; i < len; i++) {
            if (i > 0 && (len - i) % 3 == 0) result += ','; // insert a comma every 3 digits from the right
            result += raw[i];
        }
        return result;
    }

    long long countCommas(long long n) {
        long long total = 0;
        for (long long x = 1; x <= n; x++) {
            for (char c : withCommas(x)) {
                if (c == ',') total++;
            }
        }
        return total;
    }
};

int main() {
    long long n = 1002;
    Solution sol;
    cout << sol.countCommas(n) << endl; // Expected output: 3
    return 0;
}
```

■ Python

```
class Solution:
    def withCommas(self, x):
        raw = str(x)
        result = []
        length = len(raw)
        for i, ch in enumerate(raw):
            if i > 0 and (length - i) % 3 == 0:
                result.append(',') # insert a comma every 3 digits from the right
            result.append(ch)
```

```

        result.append(ch)
    return "".join(result)

def countCommas(self, n):
    total = 0
    for x in range(1, n + 1):
        total += self.withCommas(x).count(',')
    return total

if __name__ == "__main__":
    n = 1002
    print(Solution().countCommas(n))    # Expected output: 3

```

■ JavaScript

```

class Solution {
    withCommas(x) {
        const raw = String(x);
        let result = "";
        const len = raw.length;
        for (let i = 0; i < len; i++) {
            if (i > 0 && (len - i) % 3 === 0) result += ",";    // insert a comma every 3 digits from the right
            result += raw[i];
        }
        return result;
    }

    countCommas(n) {
        let total = 0;
        for (let x = 1; x <= n; x++) {
            for (const c of this.withCommas(x)) {
                if (c === ",") total++;
            }
        }
        return total;
    }
}

const n = 1002;
console.log(new Solution().countCommas(n));    // Expected output: 3

```

Complexity: Formatting each number costs $O(d)$ where d is its digit count, and there are n numbers, so this is roughly $O(n \cdot \log n)$ overall. With n up to 10^{15} , this is not merely slow — it is **physically impossible** to finish; even at a billion operations per second it would take over 30,000 years. Only useful for sanity-checking tiny inputs like the examples.

Approach 2 — Better (Formula Per Number, Skip the String Formatting)

Idea: Apply the Step 1 insight — $\text{commas}(d) = (d - 1) \div 3$ — directly per number instead of literally building a comma-formatted string. This removes all the string allocation and character scanning from Approach 1, but still visits every integer from 1 to n one at a time.

Steps

- For each x from 1 to n , compute its digit count d with a simple divide-by-10 loop (no string conversion needed).
- Add $(d - 1) \div 3$ to the running total — the exact comma count for a d -digit number.
- Return the total after the loop finishes.

Approach 2 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    // Count digits without allocating a string – a smaller constant factor than Approach 1,
    // but still one iteration per number.
    int numDigits(long long x) {
        int d = 0;
        while (x > 0) { d++; x /= 10; }
        return d;
    }

    long long countCommas(long long n) {
        long long total = 0;
        for (long long x = 1; x <= n; x++) {
            int d = numDigits(x);
            total += (d - 1) / 3;      // commas needed for a d-digit number
        }
        return total;
    }
};

int main() {
    long long n = 1002;
    Solution sol;
    cout << sol.countCommas(n) << endl;    // Expected output: 3
    return 0;
}
```

■ Python

```
class Solution:
    def numDigits(self, x):
        d = 0
        while x > 0:
            d += 1
            x //= 10
        return d

    def countCommas(self, n):
        total = 0
        for x in range(1, n + 1):
            d = self.numDigits(x)
            total += (d - 1) // 3      # commas needed for a d-digit number
        return total
```

```
if __name__ == "__main__":
    n = 1002
    print(Solution().countCommas(n))    # Expected output: 3
```

■ JavaScript

```
class Solution {
    numDigits(x) {
        let d = 0;
        while (x > 0) {
            d++;
            x = Math.floor(x / 10);
        }
        return d;
    }

    countCommas(n) {
        let total = 0;
        for (let x = 1; x <= n; x++) {
            const d = this.numDigits(x);
            total += Math.floor((d - 1) / 3);    // commas needed for a d-digit number
        }
        return total;
    }
}

const n = 1002;
console.log(new Solution().countCommas(n));    // Expected output: 3
```

Complexity: Still $O(n)$ iterations, but each one is now $O(\log x)$ arithmetic instead of $O(\log x)$ string work with allocation — a real constant-factor improvement over Approach 1, yet still fundamentally the wrong complexity class. With n up to 10^{15} this **still cannot finish** in any practical amount of time; it demonstrates the per-number formula that Approach 3 will apply to entire groups at once instead of one number at a time.

Approach 3 — Optimal (Sum Contributions Over Digit-Length Groups)

Idea: Stop looking at individual numbers entirely. Walk through digit lengths $d = 1, 2, 3, \dots$ and for each one, work out in $O(1)$ how many d -digit numbers fall inside $[1, n]$ and how many commas each of them needs, multiply, and add to the running total. Since n has at most 16 digits, this loop runs at most 16 times no matter how large n is.

Steps

- Track the current digit-length group's bounds: $\text{low} = 10^{d-1}$, $\text{high} = 10^d - 1$ (starting at $d = 1$, $\text{low} = 1$, $\text{high} = 9$).
- While $\text{low} \leq n$: clip the group's upper bound to n ($\text{upper} = \min(\text{high}, n)$), since the group might be only partially included if n itself has d digits.
- $\text{count} = \text{upper} - \text{low} + 1$ gives exactly how many d -digit numbers are $\leq n$.
- $\text{commas} = (d - 1) \div 3$ gives the comma count each of those numbers needs.
- Add $\text{count} \times \text{commas}$ to the total, then advance to the next digit-length group ($\text{low} = \text{high} + 1$, $\text{high} = \text{high} \times 10 + 9$, $d += 1$) and repeat.
- The loop naturally stops once low exceeds n — no explicit digit-count of n is even needed up front.

Approach 3 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    long long countCommas(long long n) {
        long long total = 0;
        long long low = 1, high = 9;    // range of d-digit numbers: [10^(d-1), 10^d - 1]
        int d = 1;

        while (low <= n) {
            long long upper = min(high, n);
            long long count = upper - low + 1;    // how many d-digit numbers lie within [1, n]
            long long commas = (d - 1) / 3;    // commas needed per d-digit number
            total += count * commas;

            low = high + 1;    // advance to the next digit-length group
            high = high * 10 + 9;
            d++;
        }
        return total;
    }
};

int main() {
    long long n = 1002;
    Solution sol;
    cout << sol.countCommas(n) << endl;    // Expected output: 3
    return 0;
}
```

■ Python

```
class Solution:
    def countCommas(self, n):
        total = 0
        low, high, d = 1, 9, 1    # range of d-digit numbers: [10^(d-1), 10^d - 1]

        while low <= n:
            upper = min(high, n)
```

```

        count = upper - low + 1    # how many d-digit numbers lie within [1, n]
        commas = (d - 1) // 3      # commas needed per d-digit number
        total += count * commas

        low = high + 1             # advance to the next digit-length group
        high = high * 10 + 9
        d += 1

    return total

if __name__ == "__main__":
    n = 1002
    print(Solution().countCommas(n))    # Expected output: 3

```

■ JavaScript

```

class Solution {
    countCommas(n) {
        let total = 0;
        let low = 1, high = 9, d = 1;    // range of d-digit numbers: [10^(d-1), 10^d - 1]

        while (low <= n) {
            const upper = Math.min(high, n);
            const count = upper - low + 1;    // how many d-digit numbers lie within [1, n]
            const commas = Math.floor((d - 1) / 3);    // commas needed per d-digit number
            total += count * commas;

            low = high + 1;                    // advance to the next digit-length group
            high = high * 10 + 9;
            d++;
        }
        return total;
    }
}

const n = 1002;
console.log(new Solution().countCommas(n));    // Expected output: 3

```

Complexity: $O(\log_{10} n)$ time — at most ~16 iterations for n up to 10^{15} , each doing only $O(1)$ arithmetic. $O(1)$ extra space. This is the only approach of the three that actually completes on the full constraint range, and it does so essentially instantly.

Side-by-Side Comparison

1. Brute Force	Format every number as a comma string, count commas	$O(n \log n)$ by character, never finishes	$O(\log n)$ by number
2. Better	Skip string formatting; compute commas per number from digit count	$O(n)$ by digit, never finishes at 10^{15}	$O(1)$
3. Optimal	Group numbers by digit length; sum count $\times$ commas per group	$O(\log n)$ by digit, finishes instantly	$O(1)$

Key Takeaways to Remember

- When a per-item property depends only on a coarse category (here, digit length) and not the item's exact value, stop looping item by item and instead count how many items fall in each category, then multiply.
- Digit-length ranges are always $[10^{d-1}, 10^d - 1]$ — a small, fixed set of boundaries (at most ~ 19 for any value that fits in a 64-bit integer), which is exactly why these problems collapse from $O(n)$ to $O(\log_{10} n)$.
- Whenever a constraint reaches 10^9 or beyond, treat it as a hard signal that the intended solution is a closed-form or grouped-counting formula, not a loop over every value — both brute-force tiers here are correct but **cannot possibly finish** once n approaches its stated upper bound.
- Clip each group's range to n with $\min(\text{high}, n)$ so the final, partially-included digit-length group is still counted correctly — this one line is what makes the formula work for every n , not just exact powers of ten.
- All three tiers were validated against **Example 1** ($n = 1002 \rightarrow 3$) across C++, Python, and JavaScript, and checked against Example 2 ($n = 998 \rightarrow 0$) as well.